K_1 is computably enumerable but not computable

The Open Logic Project contributors

openlogic:computability/computability-theory:local-snapshot-20261013; CC BY 4.0

Editorial note. All delivered mathematical text is the author's own, copied from the pinned Open Logic Project unit file sources/olp-computability-computability-theory/pinned-original-source.tex; no statement or proof step was rewritten, reordered or removed.

Editorial note. The counted proof is the author's complete argument: the two explain environments inside the proof (oracle intuition, reading of s(e,x,y)) are the author's own motivational commentary and are delivered verbatim; they omit nothing load-bearing

Editorial note. The label prop:k1 names the counted statement, and the proof environment immediately following it in the source is delivered verbatim as its proof.

Editorial note. The counted statement is carried by the author's own prop environment in the unit source.

Allowed standard prerequisites (editorial).

- Computable enumerability via a computable relation. Same unit, \ollabel{thm:exists-char} (source lines 1082-1088). Gives the enumerability half: K_1 = {e : exists s T(e,0,s)} is presented by a computable relation T(e,0,s).
- Universal partial computable function and the s-m-n theorem. There is a universal partial computable function Un(e,x) = phi_e(x), and the s-m-n theorem supplies a primitive recursive index s(e,x,y) with phi_{s(e,x,y)}(z) = phi_e[3](x,y,z).. Both are the constructive core: the index e with phi_e[3](x,y,z) = phi_x(y) is picked via universality, and s-m-n fixes the input x, y into the index so that halting on 0 mirrors phi_x(y).

Source reference. Original theorem thm-exists-char: sources/olp-computability-computability-theory/pinned-original-source.tex, source lines 1082–1088. Prerequisite pointer invoked in the counted proof; it names a standard fact of the same unit that the counted argument uses, and every step of the counted proof built on it is retained verbatim in the delivered body.

Proposition 0.1. *Let*

$$K_1 = \{\, e \mid \varphi_e(0)\downarrow \,\}.$$

Then K_1 is computably enumerable but not computable.

Proof. Since $K_1 = \{ e \mid \exists s\, T(e, 0, s) \}$, K_1 is computably enumerable by thm-exists-char.

To show that K_1 is not computable, let us show that K_0 is reducible to it.

This is a little bit tricky, since using K_1 we can only ask questions about computations that start with a particular input, 0. Suppose you have a smart friend who can answer questions of this type (friends like this are known as "oracles"). Then suppose someone comes up to you and asks you whether or not $\langle e, x \rangle$ is in K_0, that is, whether or not machine e halts on input x. One thing you can do is build another machine, e_x, that, for *any* input, ignores that input and instead runs e on input x. Then clearly the question as to whether machine e halts on input x is equivalent to the question as to whether machine e_x halts on input 0 (or any other input). So, then you ask your friend whether this new machine, e_x, halts on input 0; your friend's answer to the modified question provides the answer to the original one. This provides the desired reduction of K_0 to K_1.

Using the universal partial computable function, let f be the 3-ary function defined by

$$f(x, y, z) \simeq \varphi_x(y).$$

Note that f ignores its third input entirely. Pick an index e such that $f = \varphi_e[3]$; so we have

$$\varphi_e[3](x, y, z) \simeq \varphi_x(y).$$

By the *s-m-n* theorem, there is a function $s(e, x, y)$ such that, for every z,

$$\begin{aligned} \varphi_{s(e,x,y)}(z) &\simeq \varphi_e[3](x, y, z) \\ &\simeq \varphi_x(y). \end{aligned}$$

In terms of the informal argument above, $s(e, x, y)$ is an index for the machine that, for any input z, ignores that input and computes $\varphi_x(y)$.

In particular, we have

$$\varphi_{s(e,x,y)}(0)\downarrow \quad \text{if and only if} \quad \varphi_x(y)\downarrow.$$

In other words, $\langle x, y \rangle \in K_0$ if and only if $s(e, x, y) \in K_1$. So the function g defined by

$$g(w) = s(e, (w)_0, (w)_1)$$

is a reduction of K_0 to K_1. □